

```
clc;  
clear all;  
close all;
```

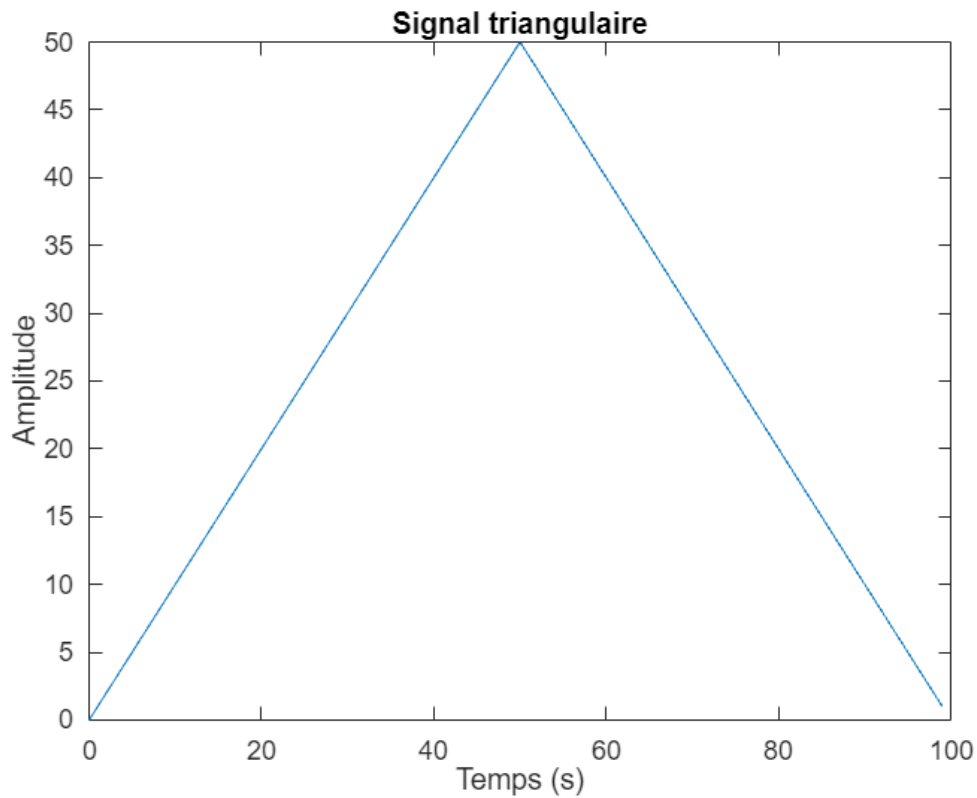
1. Construction du jeu de données

$$y[n] = (x \star h)[n] + b[n]$$

a. Simulation du signal d'entrée

$$x(t) = \begin{cases} \frac{t}{L/2} & \forall t \in [0, L/2] \\ \frac{-t + L}{L/2} & \forall t \in [L/2, L] \end{cases}$$

```
% Paramètres  
L = 100;           % duree d' observation du signal d' entree ( secondes )  
fe = 1;           % frequence d' echantillonnage (Hz)  
Te = 1/ fe ;      % periode d' echantillonnage ( secondes )  
Nx = L/ Te ;      % nombre d' echantillons du signal x  
nx = 0: Nx -1;    % indice temporel ( entier )  
tx = nx * Te ;    % base de temps ( secondes )  
  
% Signal triangulaire  
x1 = tx(1:Nx/2);   % partie croissante  
x2 = tx(Nx/2 + 1:end) - L; % partie decroissante  
x = [x1, -x2];  
  
figure;  
plot(tx, x);  
title('Signal triangulaire'); xlabel('Temps (s)'); ylabel('Amplitude');
```



b. Simulation du filtre

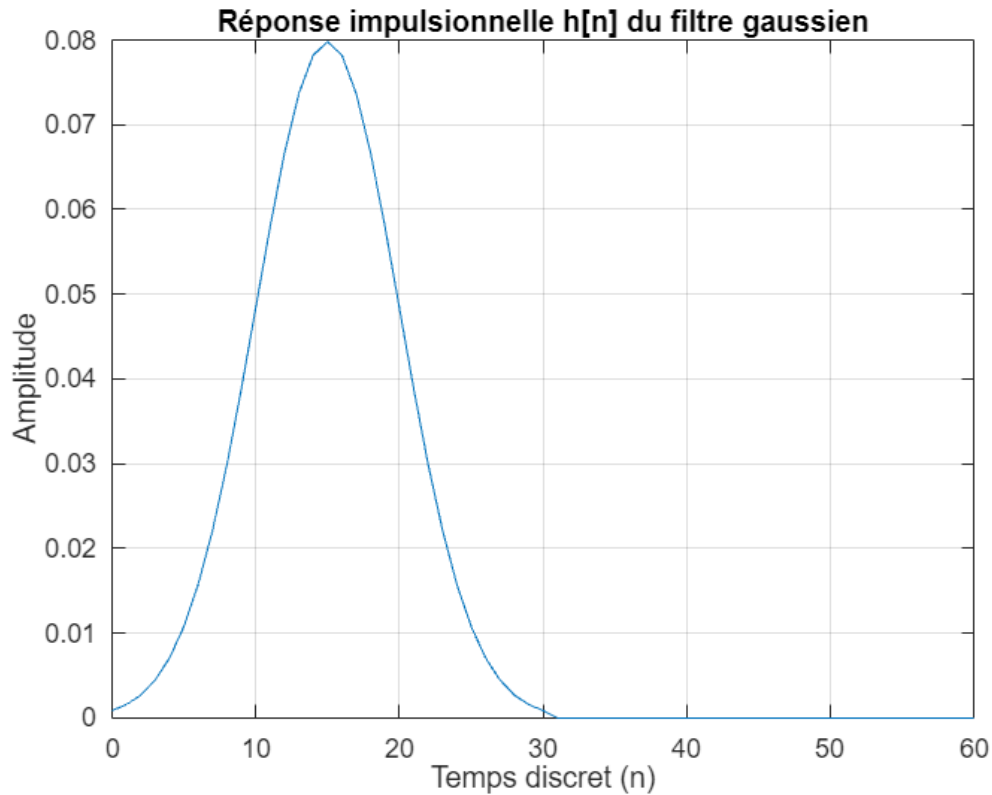
$$h(t) = \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{(t-m)^2}{2\sigma^2}\right)$$

```
%%b. Discrétisation de la réponse impulsionnelle
mh=15;
sigh=5;
%RSB=zeros(size(sigh1));
Lh=30;
Nh=2*Lh*Te +1;
% Initialisation de la réponse impulsionnelle
h = zeros(1, Nh);

% Création de la réponse impulsionnelle discrète
n = 0:Nh-1;
%for i=1:length(sigh1)
    %sigh=sigh1(i);
% Remplissage de la réponse impulsionnelle avec la forme gaussienne discrétisée
h = (1 / (sigh * sqrt(2 * pi))) * exp(-((n - mh).^2) / (2 * sigh^2));

% Troncature de la réponse impulsionnelle
h(n < 0 | n > (Lh * fe)) = 0;
```

```
% Affichage de la réponse impulsionnelle
figure;
plot(n, h);
xlabel('Temps discret (n)');
ylabel('Amplitude');
title('Réponse impulsionnelle h[n] du filtre gaussien');
grid on;
```

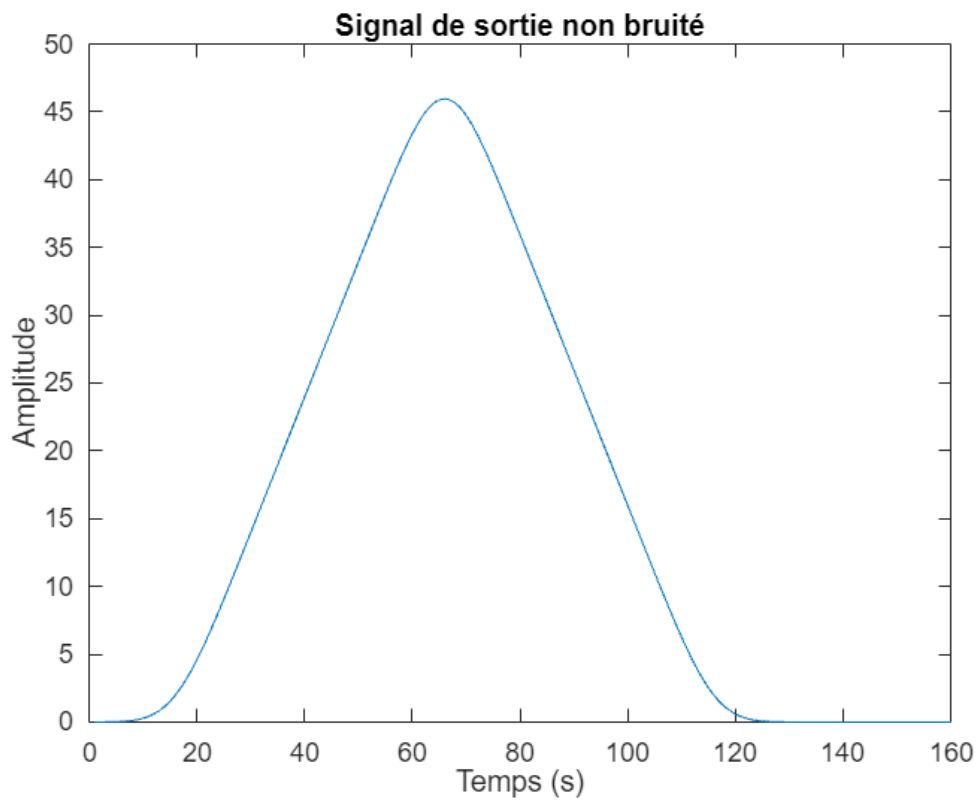


c. Simulation du problème direct

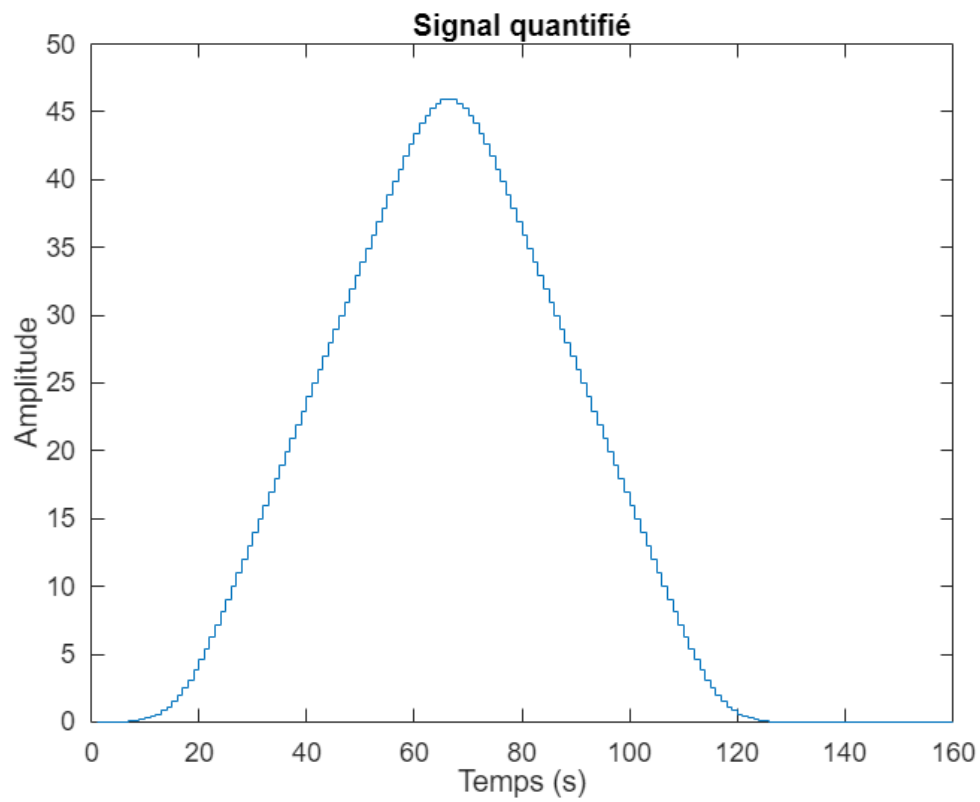
$$y[n] = (x \star h)[n] + b[n]$$

```
y_nb = conv(x, h);           % signal de sortie non bruité

figure;
plot(y_nb);
title('Signal de sortie non bruité'); xlabel('Temps (s)'); ylabel('Amplitude');
```

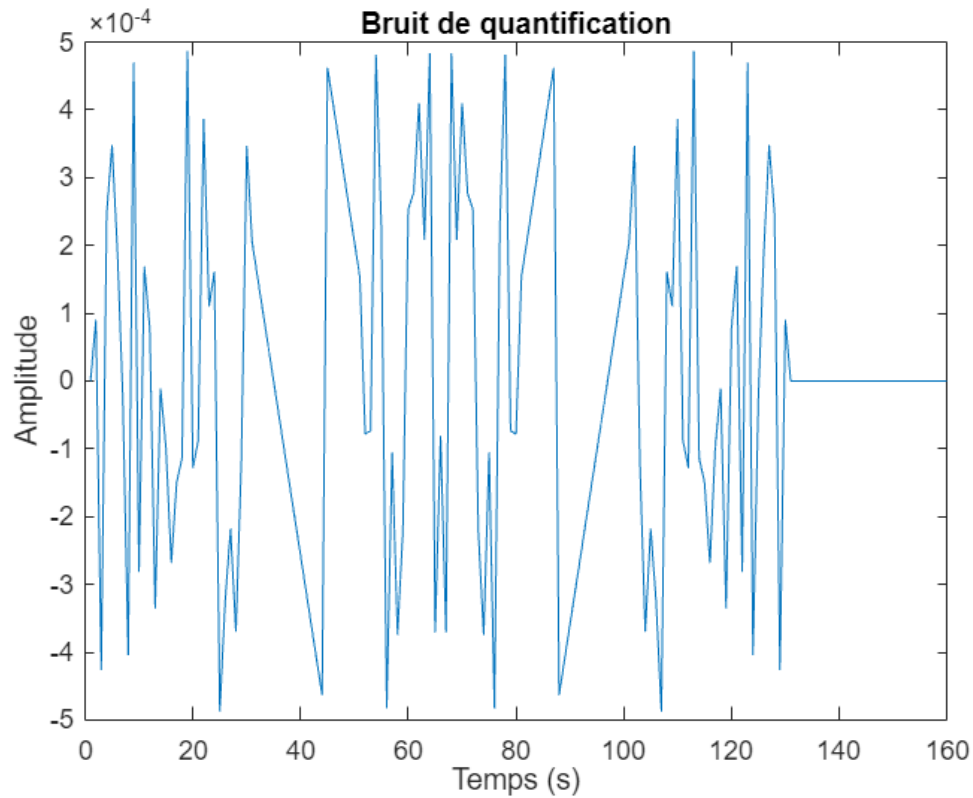


```
% niveaux de quantification  
y = round(y_nb * 2^10) / 2^10;  
  
figure;  
stairs(y);  
title('Signal quantifié'); xlabel('Temps (s)'); ylabel('Amplitude');
```



```
% Calcul et tracé du bruit de quantification
b = y - y_nb;          % bruit de quantification

figure;
plot(b);
title('Bruit de quantification'); xlabel('Temps (s)'); ylabel('Amplitude');
```



d. Simulation du problème direct par approche matricielle

$$y_{nb} = Hx$$

```

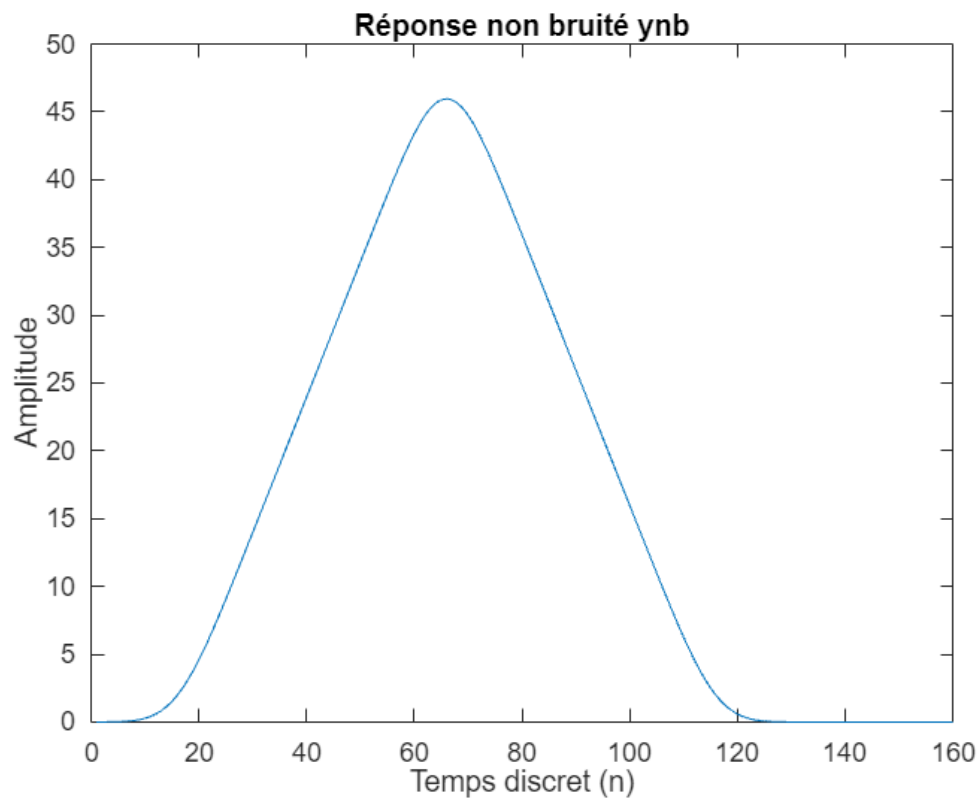
%%1. Dimension de Ny
Nh=length(h);
Nx=length(x);
Ny=Nh+Nx-1;

%%2. Construire la matrice de toeplitz H:
L1=zeros(size(x));
C1=zeros(size(y))';
C1(1:Nh,1)=h;
L1(1,1)=h(1,1);

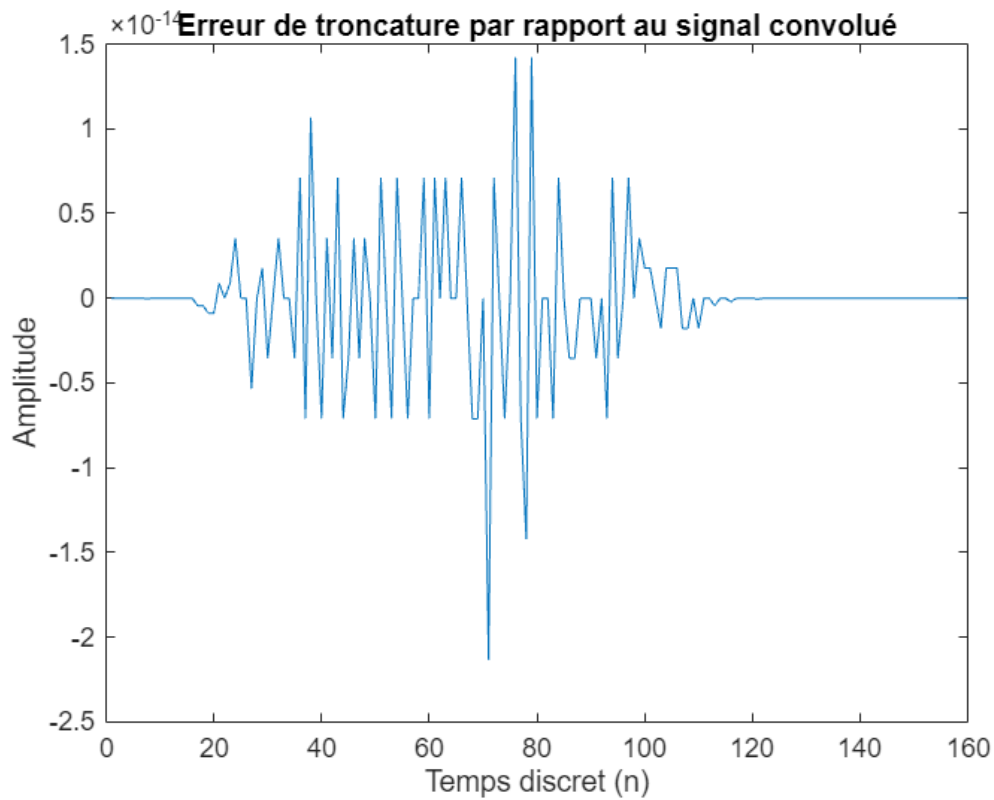
H=toeplitz(C1,L1);

%%3. Sortie non bruitée ynb par equation matricielle puis simuler l'erreur
ynb=H*x';
figure;
plot(ynb);
title('Réponse non bruité ynb');xlabel('Temps discret (n)');ylabel('Amplitude');

```



```
err_bruit = ynb - y_nb';  
figure;  
plot(err_bruit, '-');  
title('Erreur de troncature par rapport au signal convolué'); xlabel('Temps discret  
(n)'); ylabel('Amplitude');
```

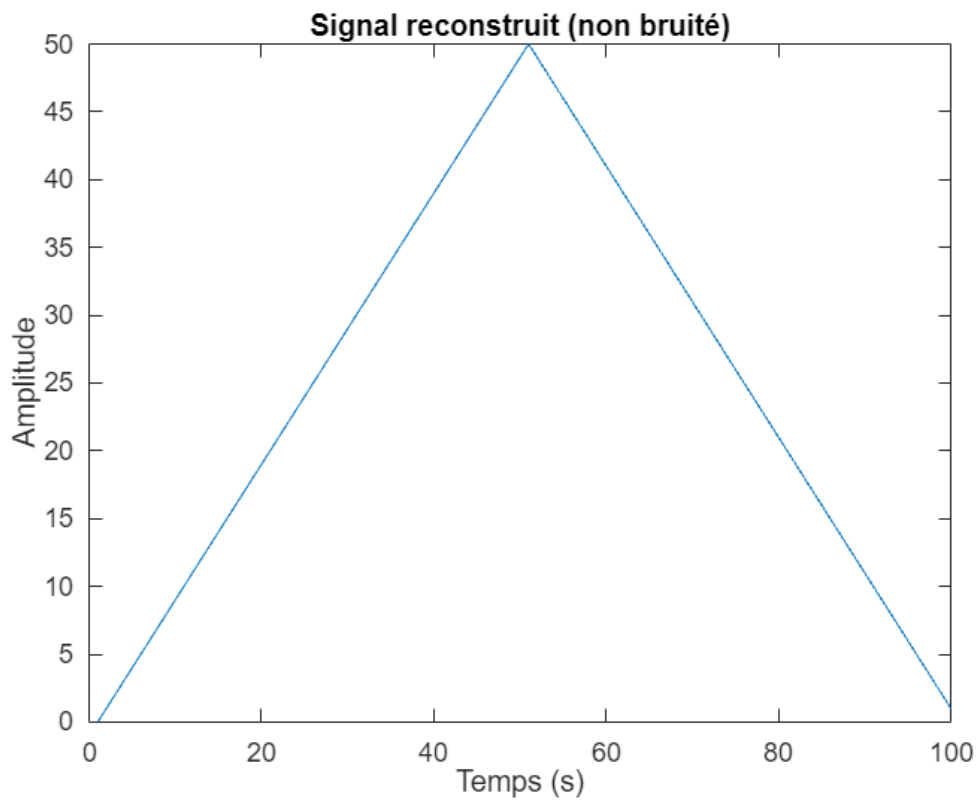


2. Déconvolution

a. Déconvolution « naïve » et constat de l'instabilité

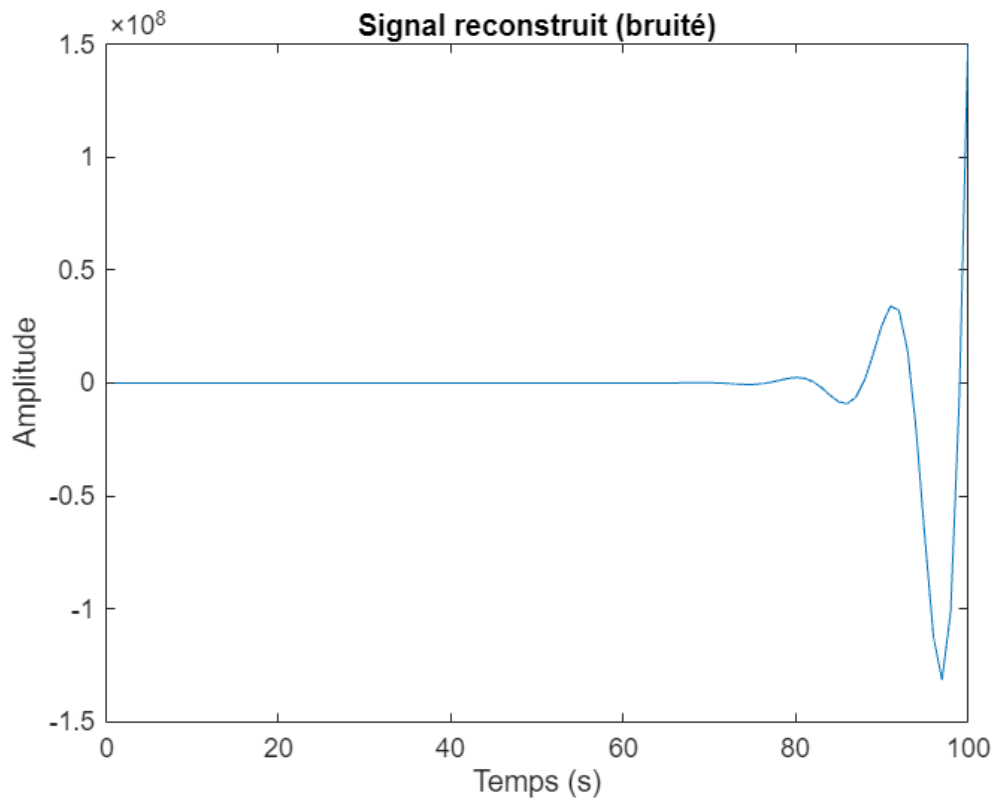
```
% Déconvolution avec signal non bruité
xrec_nb = deconv(y_nb, h);

figure;
plot(xrec_nb);
title('Signal reconstruit (non bruité)'); xlabel('Temps (s)'); ylabel('Amplitude');
```

```
% Deconvolution avec signal bruité
xrec = deconv(y, h);

figure;
plot(xrec);
title('Signal reconstruit (bruité)'); xlabel('Temps (s)'); ylabel('Amplitude');
```



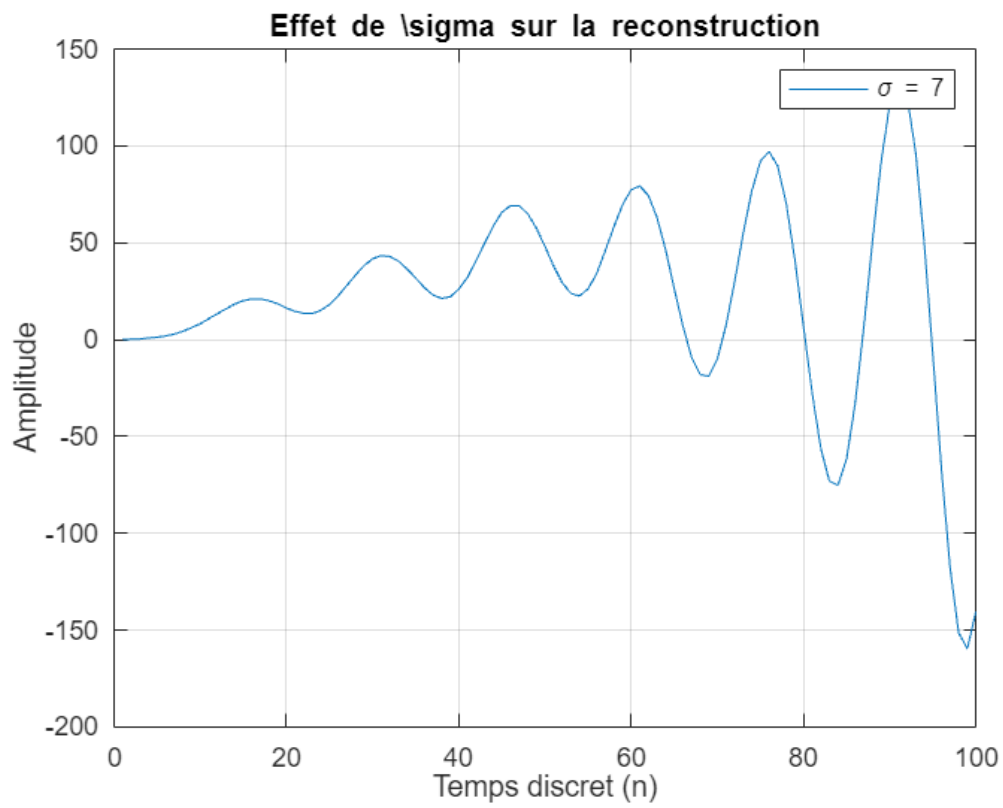
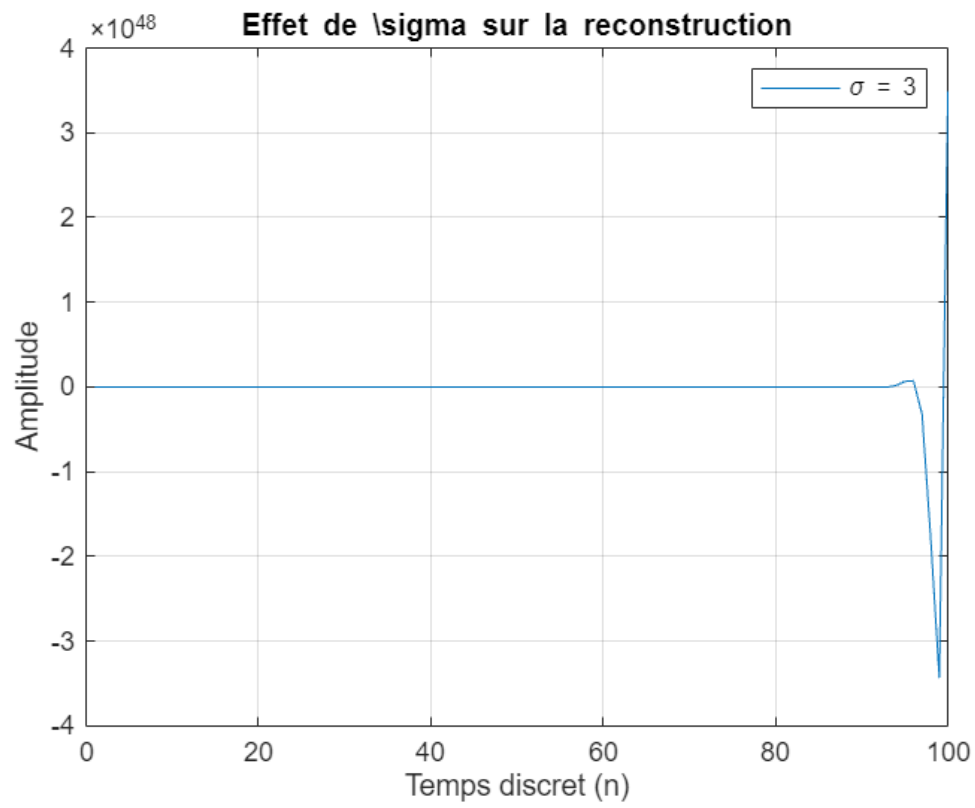
```

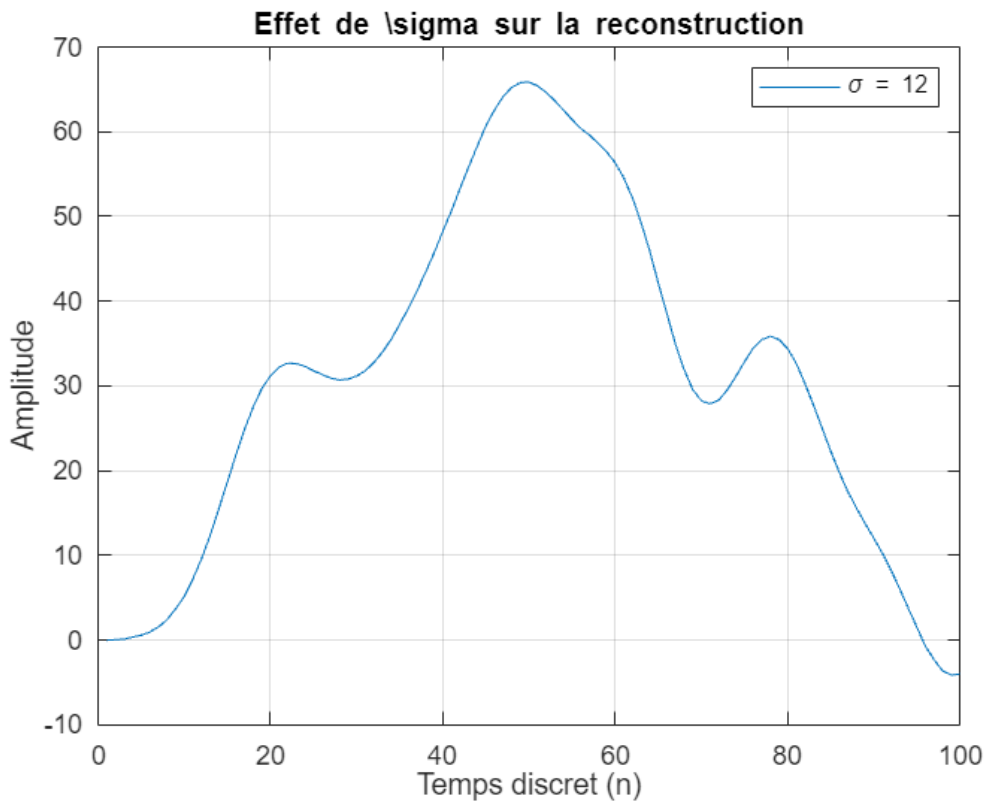
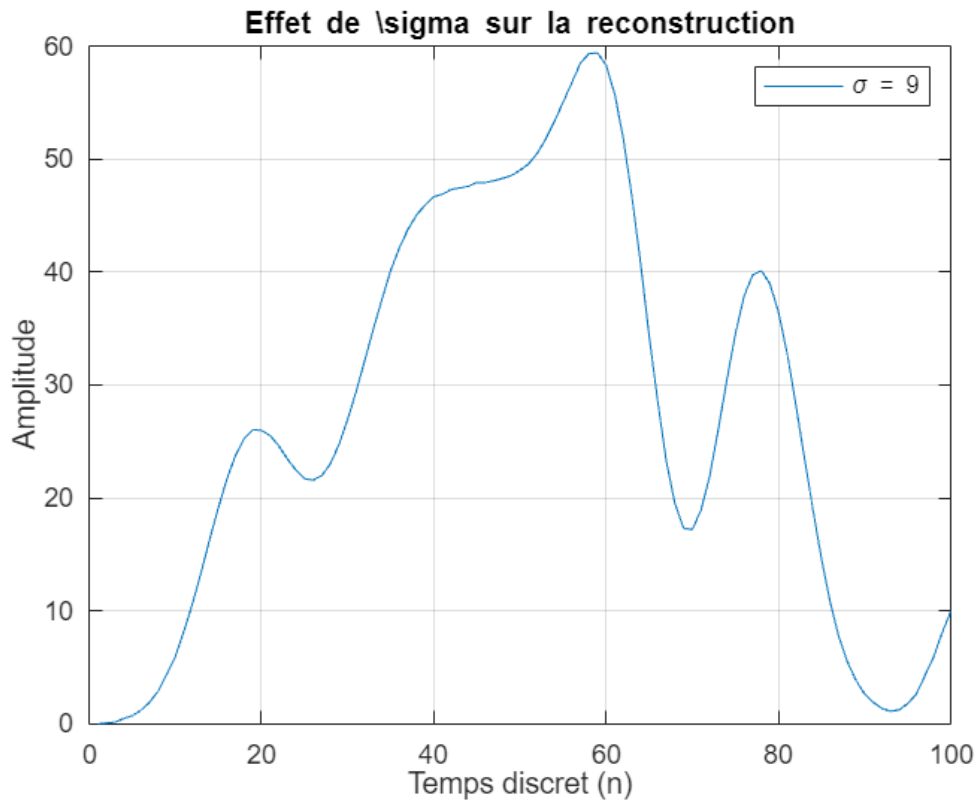
sigh_values = [3, 7, 9, 12];

for i = 1:length(sigh_values)
    % Mise à jour de sigma
    sigh = sigh_values(i);
    % Recalcul de la réponse impulsionnelle
    h = (1 / (sigh * sqrt(2 * pi))) * exp(-((n - mh).^2) / (2 * sigh^2));
    h(n < 0 | n > (Lh * fe)) = 0;
    % Déconvolution pour le signal bruité
    xrec_test = deconv(y,h);

    figure;
    plot(xrec_test, 'DisplayName', sprintf('\sigma = %d', sigh_values(i)));
    title('Effet de \sigma sur la reconstruction'); xlabel('Temps discret (n)');
    ylabel('Amplitude');
    legend; grid on;
end

```





On remarque que plus σ augmente plus le signal déconvolué ressemble au signal bruité convolué.

b. Déconvolution par moindres carrés (L2)

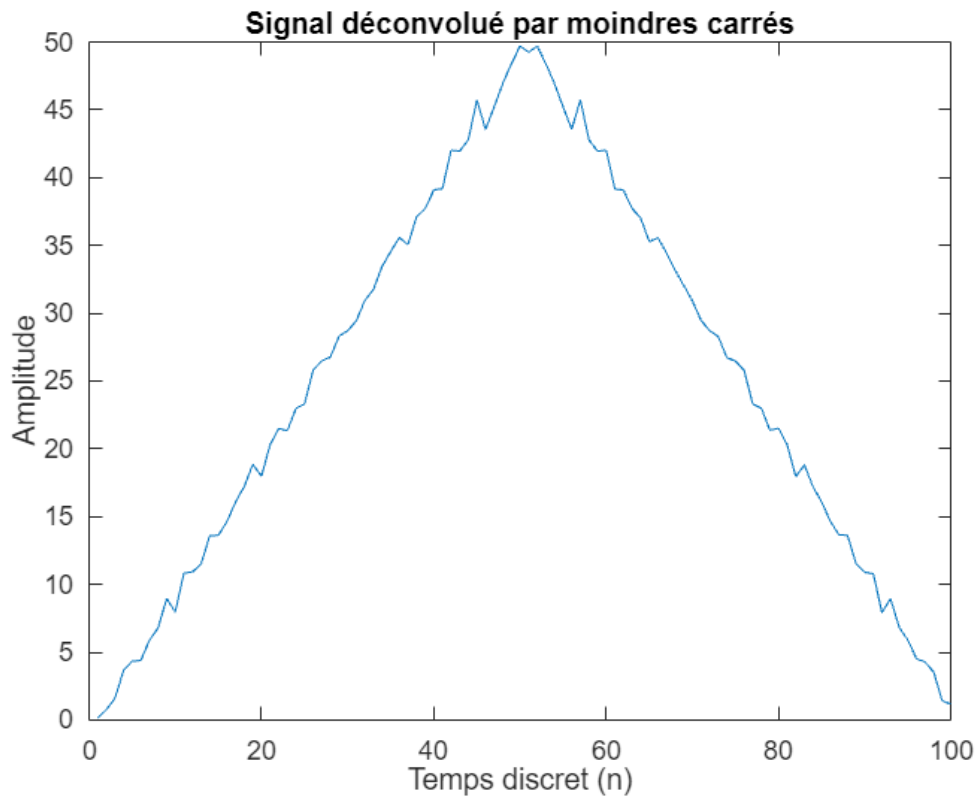
```
% Déconvolution par moindres carrés
```

```
x_lsq = (H' * H) \ (H' * y');
```

```
figure;
```

```
plot(x_lsq);
```

```
title('Signal déconvolué par moindres carrés'); xlabel('Temps discret (n)'); ylabel('Amplitude');
```



```
% Énergie de l'erreur de reconstruction
```

```
Energie_de_lerreur = sum((x' - x_lsq).^2)
```

```
Energie_de_lerreur = 38.1527
```

```
% Valeurs singulières
```

```
[U,S,V] = svd(H);
```

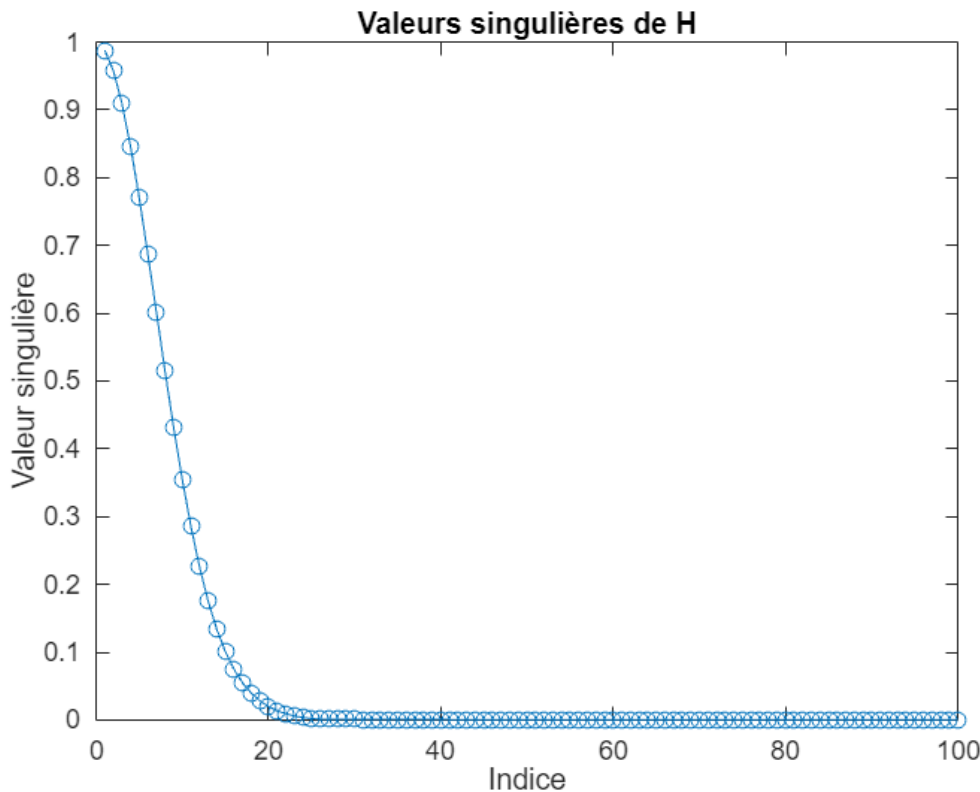
```
SVD = diag(S);
```

```
% Tracé des valeurs singulières
```

```
figure;
```

```
plot(SVD, 'o-');
```

```
title('Valeurs singulières de H'); xlabel('Indice'); ylabel('Valeur singulière');
```



```
% Calcul du conditionnement
sigma_max = max(SVD);
sigma_min = min(SVD(SVD > 0)); % Plus petite valeur singulière non nulle
cond = sigma_max / sigma_min;
disp(['Conditionnement de la matrice H : ', num2str(cond)]);
```

Conditionnement de la matrice H : 4935.0456

```
sigh_values = [3, 7, 9, 12, 15, 20];

for i = 1:length(sigh_values)
    % Mise à jour de sigma
    sigh = sigh_values(i);
    % Recalcul de la réponse impulsionnelle
    h = (1 / (sigh * sqrt(2 * pi))) * exp(-((n - mh).^2) / (2 * sigh^2));
    h(n < 0 | n > (Lh * fe)) = 0;

    %%1. Dimension de Ny
    Nh=length(h);
    Nx=length(x);
    Ny=Nh+Nx-1;

    %%2. Construire la matrice de toeplitz H:
    L1=zeros(size(x));
    C1=zeros(size(y))';
```

```

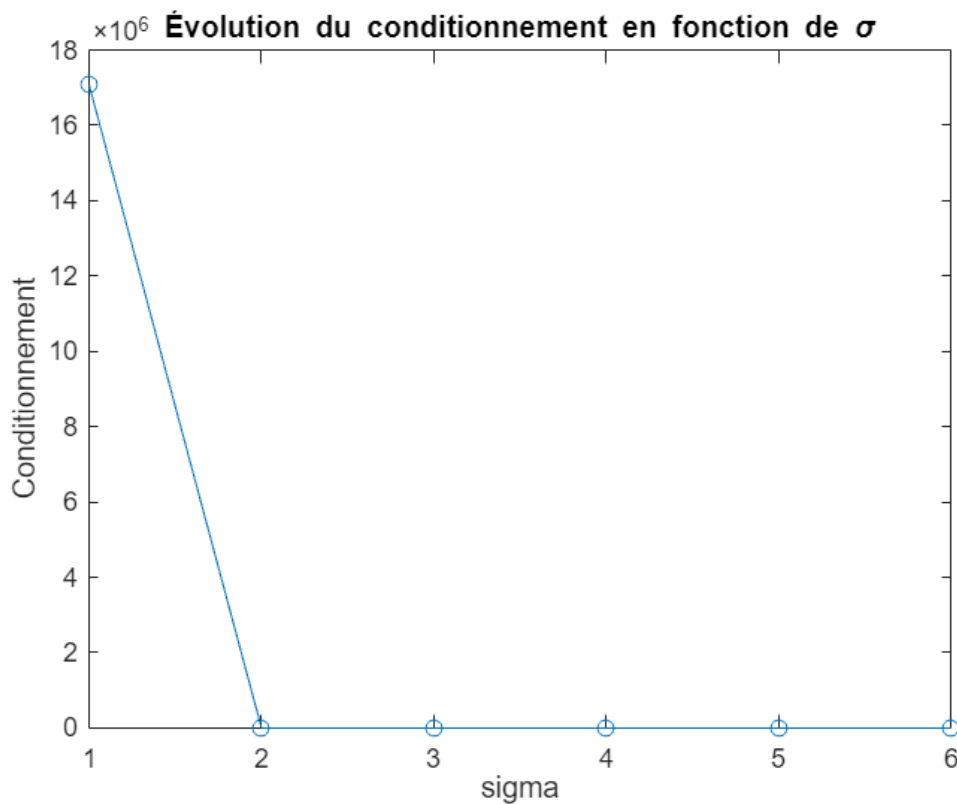
C1(1:Nh,1)=h;
L1(1,1)=h(1,1);

H=toeplitz(C1,L1);

SVD = svd(H);
sigma_max = max(SVD);
sigma_min = min(SVD(SVD > 0));
cond(i) = sigma_max / sigma_min;
end

figure;
plot(cond, '-o');title('Évolution du conditionnement en fonction de \sigma');
xlabel('sigma'); ylabel('Conditionnement');

```



La courbe montre que le conditionnement diminue brusquement avec σ . Pour des σ plus grands, le conditionnement tend vers des valeurs faibles.

c. Déconvolution par régularisation linéaire quadratique (L2L2)

$$J(x) = Fb(x) + \lambda F x(x)$$

```

% Construction de D1
dcol1 = zeros (1 , Nx ) ;
dcol1 (1:2) = dcol1 (1:2) + [1 -1];
dlig1 = zeros (1 , Nx );
dlig1 (1 ,1) = dcol1 (1 ,1) ;

```

```

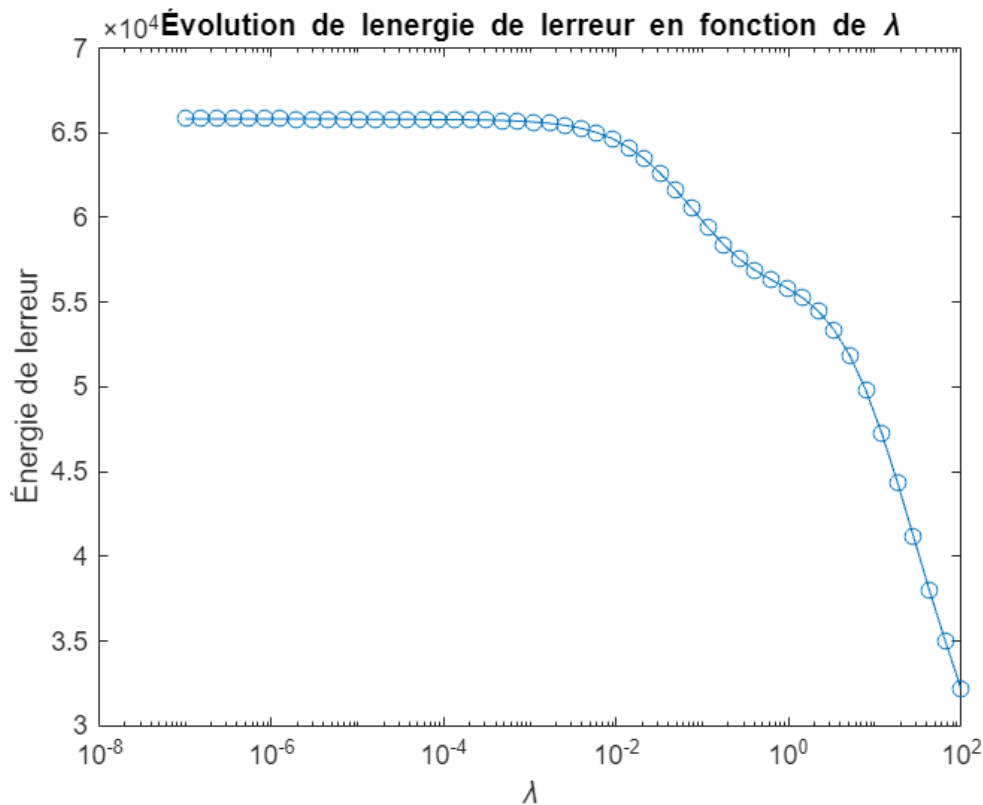
D1 = toeplitz ( dcol1 , dlig1 );

lambda_values = logspace(-7, 2); % Plage de valeurs pour lambda
errors = zeros(size(lambda_values));

for i = 1:length(lambda_values)
    lambda = lambda_values(i);
    x_reg = (H' * H + lambda * (D1' * D1)) \ (H' * y'); % Reconstruction régulière
    errors(i) = sum((x' - x_reg).^2); % Énergie de l'erreur
end

figure;
semilogx(lambda_values, errors, '-o');
title('Évolution de l'energie de lerreur en fonction de \lambda');
xlabel('\lambda');
ylabel('Énergie de lerreur');

```



```

[~, idx_min] = min(errors); % Trouver l'indice de l'erreur minimale
lambda_optimal = lambda_values(idx_min);

disp(['Valeur optimale de lambda : ', num2str(lambda_optimal)]);

```

Valeur optimale de lambda : 100

J'ai eu à me concerter avec M. Dausse et Mme Ouchabane par rapport à l'élaboration de la réponse impulsionnelle. Je me suis aussi rapproché d'ancien promotionnaire, Mme Kaire et M. Lam qui ont su me guider pour parfaire mon code.

Je vous en souhaite bonne réception.

```
export("TP_Optimisation_TOURE.mlx", format ="pdf");
```